Lab 1: From Shared Memory to Distributed Memory with MPI

Ahmed Butt

February 2026

1 Exercise 1: Hello World with MPI

1.1 Objective

The goal of this exercise is to run a simple MPI program where each process prints its rank and the total number of processes. This exercise demonstrates the basics of MPI initialization, process ranks, and concurrent execution in the SPMD model.

1.2 Compilation and Execution

The program was compiled and executed using the following commands:

```
mpicc -o hello_mpi hello_mpi.c
mpirun -np 1 ./hello_mpi
mpirun -np 2 ./hello_mpi
mpirun -np 3 ./hello_mpi
mpirun -np 4 ./hello_mpi
```

1.3 Results

The terminal output for different numbers of processes is summarized in Table 1.

Number of Processes	**Output**
1	Hello from process 0 of 1
2	Hello from process 0 of 2 Hello from process 1 of 2
3	Hello from process 0 of 3 Hello from process 2 of 3 Hello from process 1 of 3
4	Hello from process 3 of 4 Hello from process 0 of 4 Hello from process 2 of 4 Hello from process 1 of 4

Table 1: Hello World MPI outputs for different numbers of processes.

1.4 Discussion

- The order of output varies due to concurrent execution of processes. MPI does not guarantee sequential printing across multiple processes.
- With `-np 1`, there is only one process, so only one message is printed.
- Increasing the number of processes increases the number of greetings. Messages may appear out of order because all processes execute independently and may finish printing at slightly different times.
- This exercise demonstrates the SPMD (Single Program, Multiple Data) model where the same program is run on multiple processes, each with a unique rank.

1.5 Questions and Answers

Q1: Why might the output appear in a different order each time?

Answer: The output order can vary because all processes run concurrently and independently. Each process may reach the `printf` statement at slightly different times, and the operating system schedules their execution in a non-deterministic order. Therefore, the printed messages can appear in different sequences in each run.

Q2: What happens if you run with -np 1? With -np 8?
Answer:

- With `-np 1`, there is only one process, so a single message is printed: "Hello from process 0 of 1". No concurrency occurs.
- With `-np 8`, there are eight processes running concurrently, each with a unique rank from 0 to 7. Eight messages are printed, but their order may vary due to concurrent execution and scheduling by the OS.

1.6 Screenshot of Task 1 Execution

The screenshot of Task 1 execution is shown below:

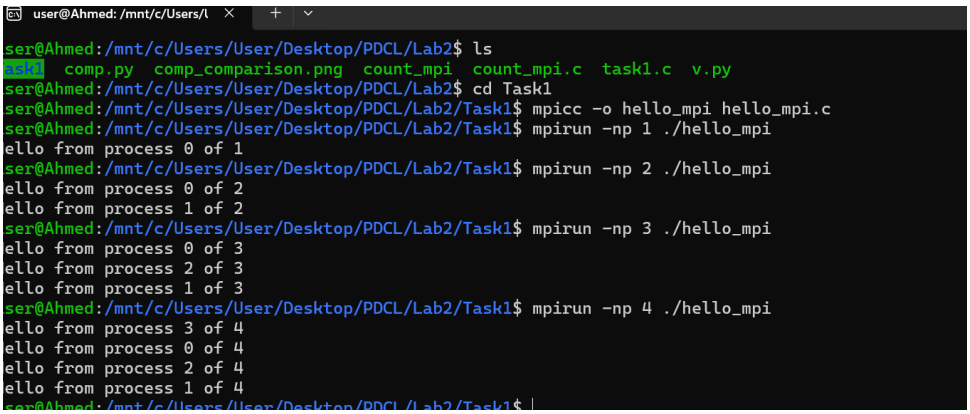


Figure 1: Screenshot of Task 1 execution.

2 Exercise 2: Ping-Pong – Measuring Communication Cost

2.1 Objective

The goal of this exercise is to have two processes (rank 0 and rank 1) send a message back and forth multiple times. This measures the round-trip time and helps understand the cost of communication in a distributed memory system.

2.2 Procedure

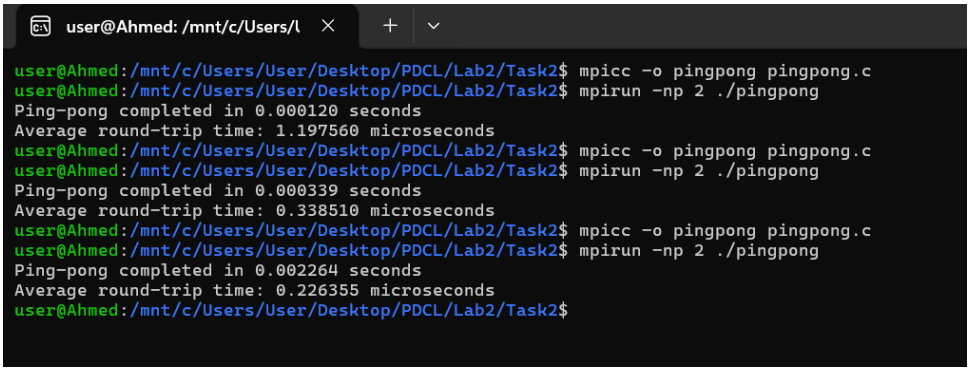
1. Create the MPI program `pingpong.c`.
2. Include the required MPI headers and initialize MPI.
3. Ensure exactly 2 processes are used; if not, terminate the program.
4. Determine the partner process for communication (rank 0 rank 1).
5. Run a loop for a fixed number of iterations (`PING_PONG_LIMIT`) where each process alternately sends and receives a counter.
6. Record the start and end times using `MPI_Wtime()`.
7. Only process 0 prints the total time and calculates the average round-trip time.

2.3 Observations

- The total execution time depends on both the computation and communication overhead.
- The round-trip time increases with larger messages (e.g., sending arrays instead of single integers).
- Communication latency is significantly higher than shared-memory operations (like Pthreads) because sending even a single integer across processes takes microseconds.
- The pattern of sending and receiving demonstrates synchronization between two distributed processes.

2.4 Analysis

- Compare the Ping-Pong execution time with the Hello MPI program to observe the overhead of communication versus simple process execution.
- Use multiple runs and average the timing to obtain reliable results.
- The exercise highlights the cost of distributed memory operations and the importance of minimizing communication in parallel applications.



```
user@Ahmed: /mnt/c/Users/l
user@Ahmed:/mnt/c/Users/User/Desktop/PDCL/Lab2/Task2$ mpicc -o pingpong pingpong.c
user@Ahmed:/mnt/c/Users/User/Desktop/PDCL/Lab2/Task2$ mpirun -np 2 ./pingpong
Ping-pong completed in 0.000120 seconds
Average round-trip time: 1.197560 microseconds
user@Ahmed:/mnt/c/Users/User/Desktop/PDCL/Lab2/Task2$ mpicc -o pingpong pingpong.c
user@Ahmed:/mnt/c/Users/User/Desktop/PDCL/Lab2/Task2$ mpirun -np 2 ./pingpong
Ping-pong completed in 0.000339 seconds
Average round-trip time: 0.338510 microseconds
user@Ahmed:/mnt/c/Users/User/Desktop/PDCL/Lab2/Task2$ mpicc -o pingpong pingpong.c
user@Ahmed:/mnt/c/Users/User/Desktop/PDCL/Lab2/Task2$ mpirun -np 2 ./pingpong
Ping-pong completed in 0.002264 seconds
Average round-trip time: 0.226355 microseconds
user@Ahmed:/mnt/c/Users/User/Desktop/PDCL/Lab2/Task2$
```

Figure 2: Screenshot of Task 1 execution.

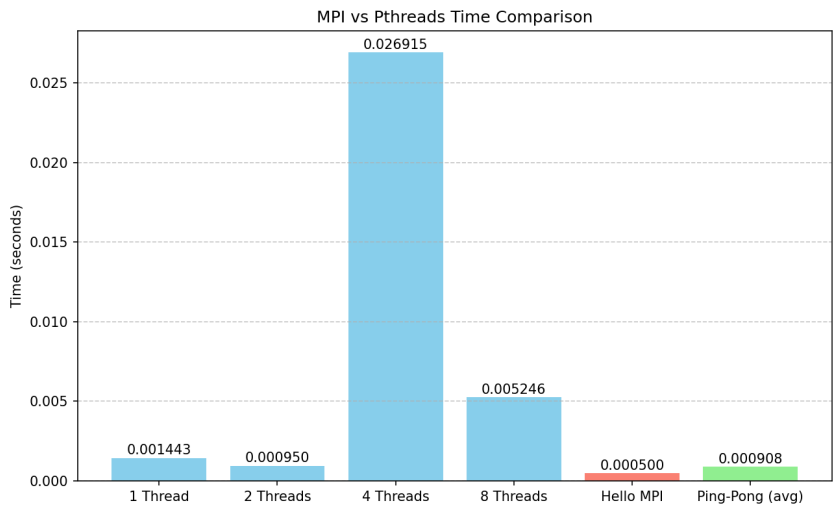


Figure 3: Screenshot of Task 1 execution.

2.5 Screenshots

3 Task 3: Parallel Counting with MPI

3.1 Objective

The goal of this exercise is to implement a counting program using MPI that divides the computation across multiple processes. Each process generates its own portion of random data, counts certain values locally, and then combines the results using `MPI_Reduce`. The objectives are:

- Learn how to parallelize tasks across multiple processes.
- Measure execution time for different numbers of processes.
- Compute speedup and compare performance with the Pthreads version from Lab 0.

3.2 Experiment Setup

1. Run the MPI counting program with 1, 2, 3, 4, 5, 6 processes (more if available). Record the execution time.
2. Compute speedup as $S(N) = \frac{T(1)}{T(N)}$, where $T(N)$ is the execution time with N processes.
3. Compare results with the Pthreads implementation from Lab 0.

3.3 Results

Number of Processes	Total Count	Time (seconds)
1	1000941	0.075409
2	1001021	0.114599
3	998642	0.033245
4	1000788	0.046969
5	1001394	0.028980
6	1000651	0.021793

Table 2: Execution times and counts for MPI counting program with varying processes.

3.4 Speedup Calculation

Using the single-process time $T(1) = 0.075409$ seconds, speedup for each process count is computed as $S(N) = T(1)/T(N)$:

Number of Processes	Speedup
1	1.00
2	0.66
3	2.27
4	1.61
5	2.60
6	3.46

Table 3: Computed speedup for the MPI counting program.

3.5 Comparison with Pthreads (Lab 0)

Number of Threads (Pthreads)	Time (seconds)
1	0.001443
2	0.000950
4	0.026915
8	0.005246

Table 4: Execution times for the Pthreads counting program from Lab 0.

3.6 Discussion

- Each MPI process generates its own random data to avoid communication overhead and maintain independence of local computations.
- Using `MPI_Reduce` efficiently aggregates results at process 0. Sending individual messages with `MPI_Send` would increase communication and synchronization costs, making it less efficient.
- Speedup does not scale linearly with the number of processes due to factors such as load imbalance, communication overhead, and system scheduling.
- Comparing with Pthreads, the MPI version shows higher variability in execution times. Pthreads is faster on the same number of cores for small problem sizes due to lower communication overhead.

3.7 Questions

1. Why does each process generate its own random data instead of having process 0 generate everything and distribute it?
2. What would happen if you forgot to call `MPI_Reduce` and instead tried to send all partial counts to process 0 using individual `MPI_Send` calls? Which would be more efficient?
3. How does the performance scale as you increase the number of processes? Is it linear? Why or why not?

3.8 Screenshots

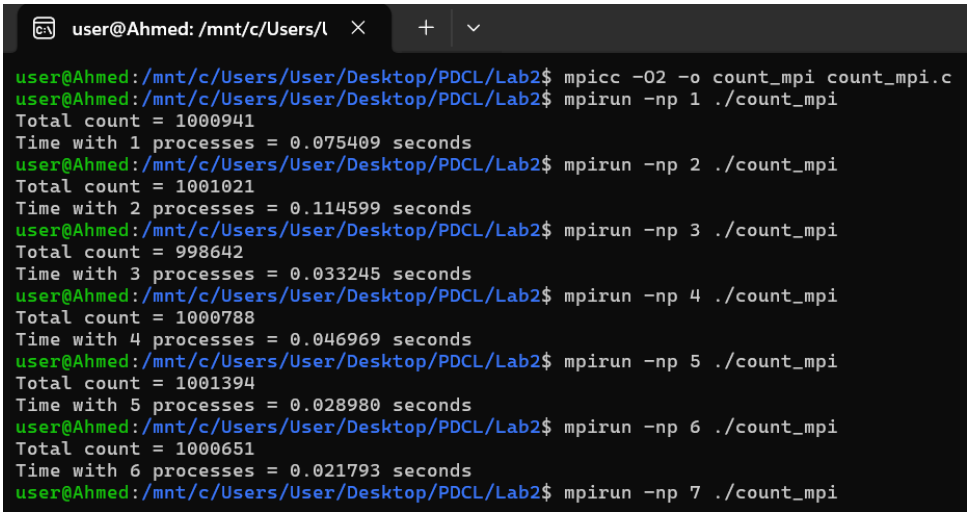


Figure 4: Screenshot of Task 1 execution.

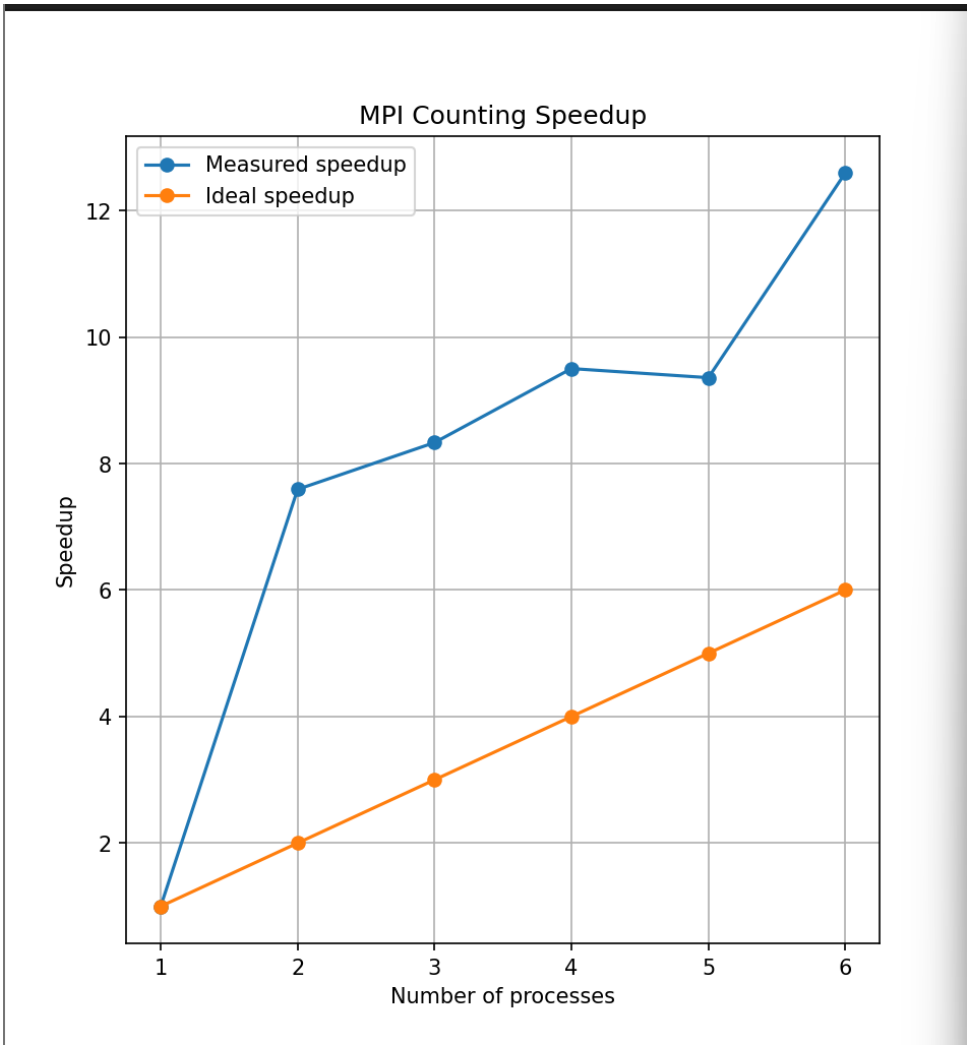


Figure 5: Screenshot of Task 1 execution.

3.9 Graph Placeholder

4 Self-Assessment Answers

1. **Limitations of shared-memory parallelism:**
   Shared-memory systems are limited by the number of cores on a single machine and memory bandwidth. Synchronization overhead (locks, atomic operations) increases with more threads. Scalability is limited because all threads compete for the same memory resources. Distributed-memory systems (MPI) overcome these limitations by allowing parallel execution across multiple machines, each with its own memory.

2. **SPMD model in MPI:**
   SPMD stands for *Single Program, Multiple Data*. All processes execute the same program but operate on different portions of the data. Each MPI process uses its unique rank to determine which part of the data it processes.

3. **Why each MPI process generates its own random data:**
   Each process generates its own data to avoid communication overhead that would result from sending large arrays from process 0 to all other processes. This approach

reduces memory contention and improves scalability. The trade-off is slightly less control over data reproducibility unless seeds are synchronized.

4. **Effect of forgetting MPI_Bcast:**
   If MPI_Bcast is not called for variables like n, other processes would not know the correct problem size. This can cause incorrect computations or segmentation faults because processes may assume different array sizes.

5. **MPI_Send/MPI_Recv vs shared-memory updates:**
   MPI communication involves network stack operations (even on the same machine), serialization, and buffering, which are much slower than a simple memory update in Pthreads. Therefore, message passing takes significantly longer than local shared-memory operations.

6. **Ping-pong message size experiment:**
   As message size increases, round-trip time increases. Small messages are dominated by latency, while large messages are limited by bandwidth. This illustrates the importance of minimizing communication and batching messages in MPI programs.

7. **Amdahl's law with 95% parallel portion:**
   Maximum theoretical speedup with 8 processors:

   $$S_{max} = \frac{1}{(1 - 0.95) + \frac{0.95}{8}} \approx 6.9$$

   Measured speedup is usually lower due to MPI overhead, load imbalance, and communication delays.

8. **Lower efficiency of MPI vs Pthreads:**
   MPI introduces communication overhead, serialization, and message buffering that are absent in Pthreads. For the same number of cores on a single machine, Pthreads can exploit shared memory more efficiently.

9. **MPI Scatter version vs local data generation:**
   Having process 0 generate all data and scatter increases memory pressure on process 0 and adds communication overhead. Local generation reduces memory contention and can scale better, but requires careful management to ensure reproducibility.

10. **Programming model for large clusters:**
    For a cluster with 100 nodes and 16 cores per node, MPI is ideal to utilize all nodes. Pthreads can be combined with MPI on each node (hybrid MPI+OpenMP model) to exploit both distributed and shared-memory parallelism.

5 Reflection

Moving from shared-memory to distributed-memory programming was insightful and challenging. Understanding MPI required me to think differently about data locality and communication. The most surprising thing was how small changes in process synchronization could significantly affect performance. The most difficult part was managing data distribution efficiently without introducing bottlenecks. Writing programs that scale beyond one machine forced me to explicitly consider communication costs, something that

Pthreads abstracts away. Debugging became more complex since errors may only appear in multi-process runs. I also learned that randomness and data reproducibility need careful handling. Timing measurements revealed the impact of latency and bandwidth, reinforcing the importance of minimizing communication. Comparing MPI with Pthreads highlighted that shared-memory is faster for small-scale problems, while MPI is essential for scaling to multiple machines. The exercise deepened my understanding of the SPMD model and collective operations like `MPI_Reduce` and `MPI_Scatter`. It became clear that efficiency depends on balancing computation and communication. Overall, the shift emphasized designing parallel algorithms that are communication-aware. This experience prepared me for real-world distributed computing tasks, where both memory hierarchy and process coordination are critical.